7. Artificial Neural Networks

Machine learning

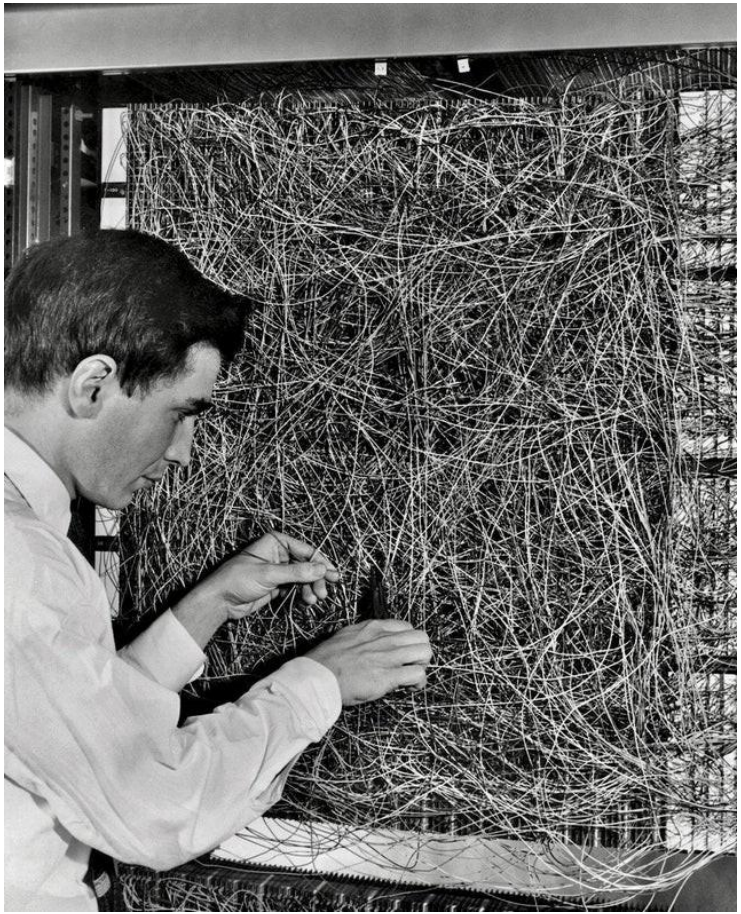
Jiri Chmelik
Department of Biomedical Engineering

Introduction

- Basics
  - principle of neural nets
  - single neuron
    - principle
    - activation functions
    - derivation
  - general neural net
    - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
  - hyperparameter optimization

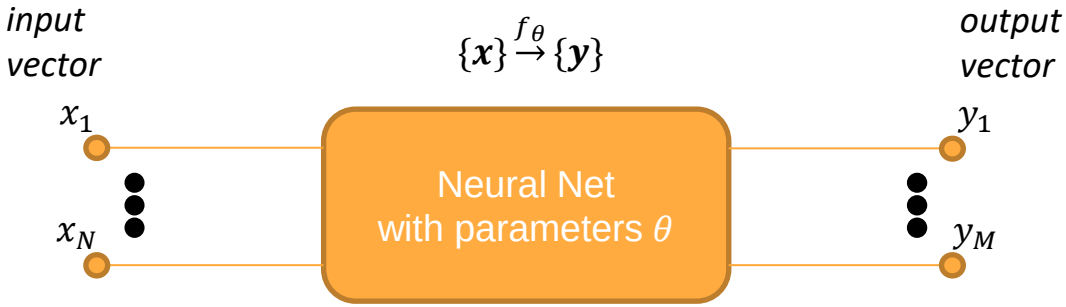
Basics

- Basics
  - principle of neural nets
  - single neuron
    - principle
    - activation functions
    - derivation
  - general neural net
    - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
  - hyperparameter optimization



Frank Rosenblatt and his Mark I Perceptron

Principle of Neural Nets



$$\hat{\boldsymbol{y}} = f_\theta(\boldsymbol{x})$$

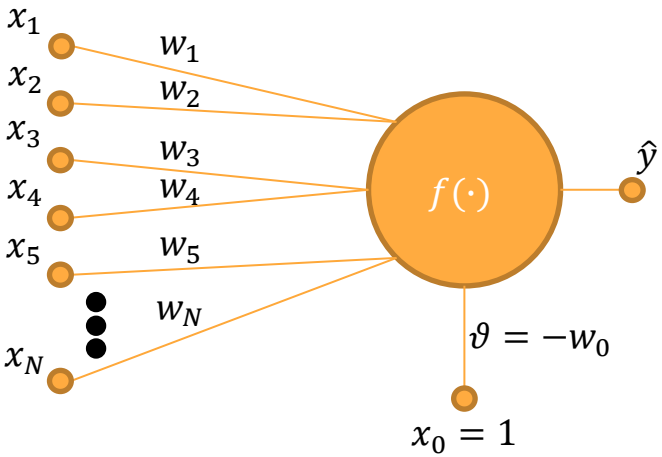
Problem: find optimal parameters θ

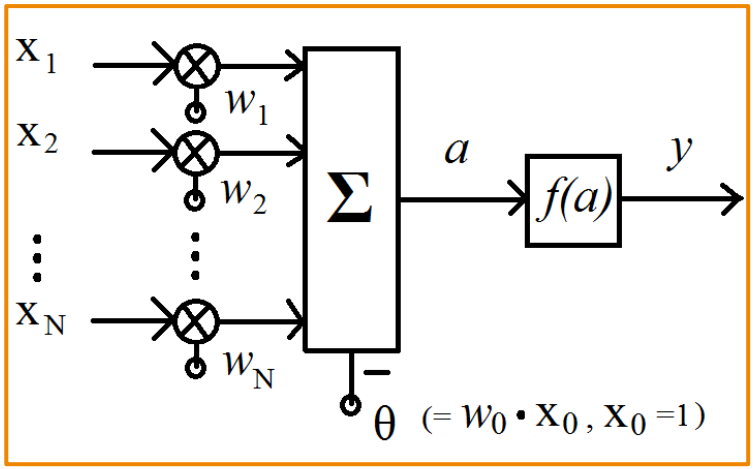
$$minimize_\theta \quad L(y, f_\theta(\boldsymbol{x}))$$ L – error/loss function

Solution: Gradient descent

$$\theta^{it+1} = \theta^{it} - \mu \nabla_\theta L(y, f_\theta(\boldsymbol{x}))$$ μ – learning rate/step

Artificial neuron





Output: $y = f\left[\sum_{i=1}^{N} w_i x_i - \vartheta\right] = f\left[\sum_{i=0}^{N} w_i x_i\right]$, *were* $x_0 = 1,\ w_0 = -\vartheta$

activation

activation function

inner potential

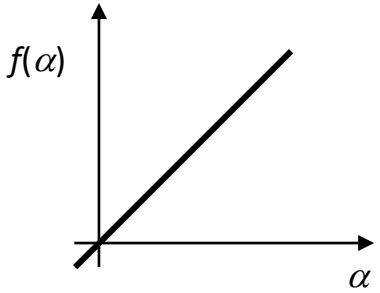
threshold

weights

input

Activation functions

- **Linear**



$f(\alpha) = \alpha \qquad f(\alpha) \in \mathbb{R}$

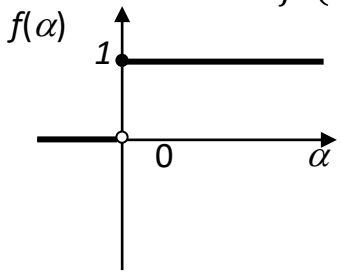
$f'(\alpha) = 1$

- **Step functions**

Heaviside step function (unit step function):

$$f(\alpha) = \frac{sign(\alpha) + 1}{2} \qquad f(\alpha) \in \{0, 1\}$$

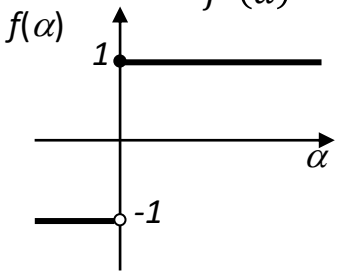
$$f'(\alpha) = \delta(\alpha)$$



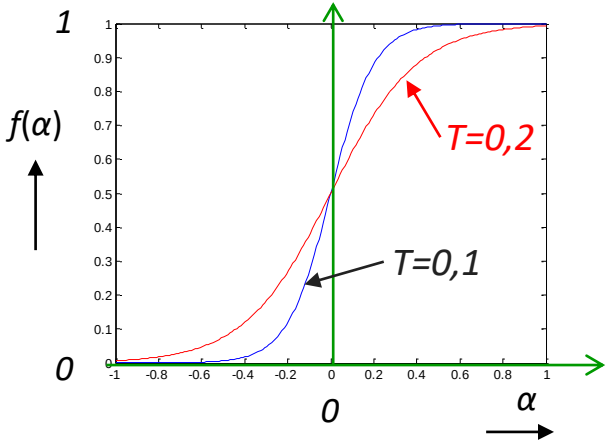
Signum:

$f(\alpha) = sign(\alpha) \qquad f(\alpha) \in \{-1, 1\}$

$f'(\alpha) = \delta(\alpha)$



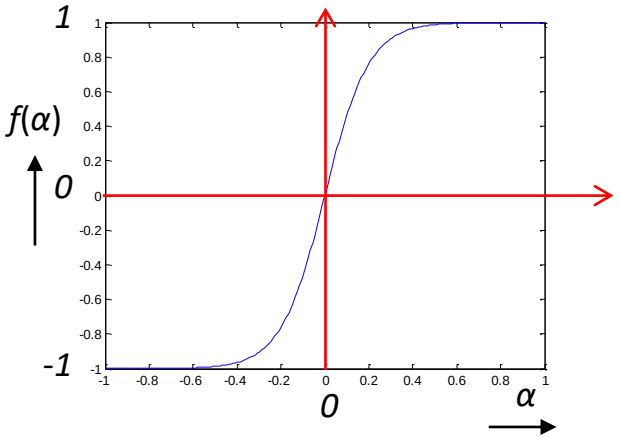
- **Sigmoid**



$$\sigma(\alpha) = f(\alpha) = \frac{1}{1 + e^{\frac{-\alpha}{T}}} \qquad f(\alpha) \in [0,\ 1]$$

$$f'(\alpha) = f(\alpha)(1 - f(\alpha))$$

- **Hyperbolic tangent (tanh)**

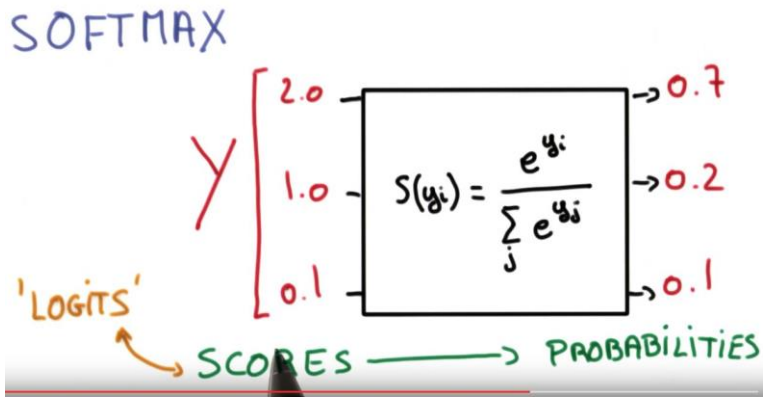


$$f(\alpha) = 2\sigma(2\alpha) - 1 = 2\frac{1}{1 + e^{-2\alpha}} - 1 = \frac{2}{1 + e^{2\alpha}} - 1$$

$$= \frac{e^{2\alpha} - 1}{e^{2\alpha} + 1} \qquad f(\alpha) \in [-1,\ 1]$$

$$f'(\alpha) = 4f(\alpha)(1 - f(\alpha))$$

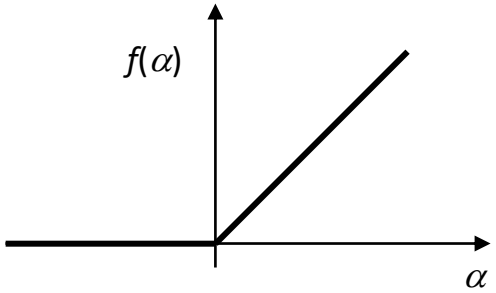
- **Softmax**



$$f(\alpha_i) = \frac{e^{\alpha_i}}{\sum_{j=1}^{K} e^{\alpha_j}} \qquad f(\alpha_i) \in [0,\ 1],\ \sum_{j=1}^{K} f(\alpha_j) = 1$$

$$f'(\alpha_i) = \begin{cases} f(\alpha_i)\left(1 - f(\alpha_j)\right) & if\ i = j \\ -f(\alpha_i) f(\alpha_j) & if\ i \neq j \end{cases}$$

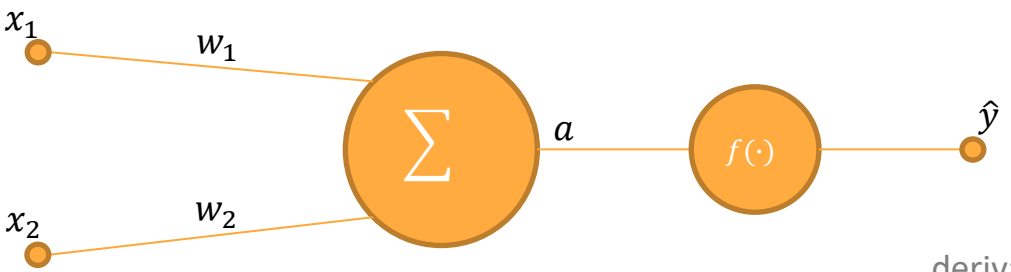
- **ReLU (Rectified Linear Unit)**



$$f(\alpha) = \max(0, \alpha) \quad f(\alpha) \in [0, \infty)$$

$$f'(\alpha) = \begin{cases} 0\ if\ \alpha < 0 \\ 1\ if\ \alpha > 0 \end{cases}$$

Derivation of single neuron



$$D = \{(\boldsymbol{x}^{(1)}, y^{(1)}), \dots, (\boldsymbol{x}^{(N)}, y^{(N)})\}$$

loss function

$$L = \frac{1}{2}(\hat{y} - y)^2$$

derivation of loss function
(for $f(\cdot)$ – linear activation function)

$$L = \frac{1}{2}(x_1 w_1 + x_2 w_2 - y)^2$$

derivation of loss function
(for $f(\cdot)$ – sigmoid activation function)

$$L = \frac{1}{2}\left(\frac{1}{1 + e^{-(x_1 w_1 + x_2 w_2)}} - y\right)^2$$

$$\frac{dL}{dw_i} = 2\frac{1}{2}(x_1 w_1 + x_2 w_2 - y)x_i$$

$$\frac{dL}{dw_i} = 2\frac{1}{2}[f(x_1 w_1 + x_2 w_2) - y][f(x_1 w_1 + x_2 w_2)\big(1 - f(x_1 w_1 + x_2 w_2)\big)]x_i$$

generalized derivation for any loss function

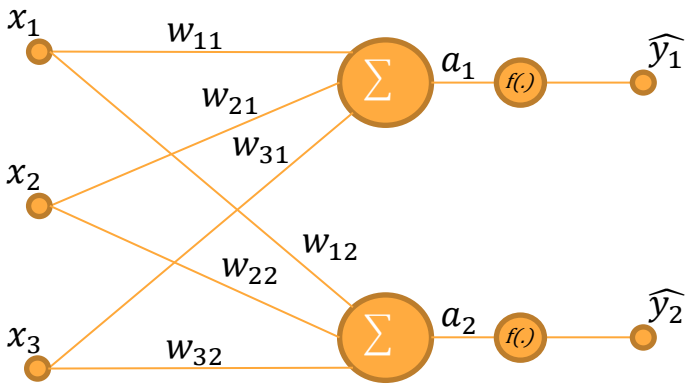
$$\frac{dL}{dw_i} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{da} \cdot \frac{da}{dw_i}$$

$$\frac{dL}{d\hat{y}} = 2\frac{1}{2}(\hat{y} - y) \cdot 1 = (\hat{y} - y)$$ loss fcn. dependent

$$\frac{d\hat{y}}{da} \rightarrow dependent\ on\ used\ activation\ fcn.$$

$$\frac{da}{dw_i} = x_i$$

Matrix syntax



$$\begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{21} & w_{31} \\ w_{12} & w_{22} & w_{32} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$\boldsymbol{a} = \boldsymbol{W}\,\boldsymbol{x}$$

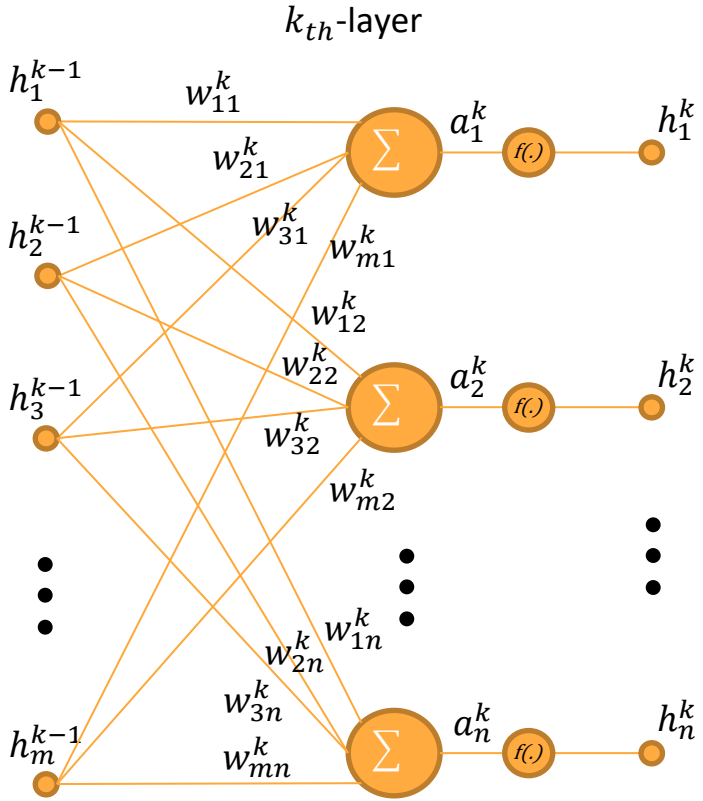
$$\hat{\boldsymbol{y}} = f(\boldsymbol{a}) = \begin{bmatrix} f(a_1) \\ f(a_2) \end{bmatrix}$$

loss function

$$\boldsymbol{L} = \frac{1}{2}(\hat{\boldsymbol{y}} - \boldsymbol{y})^2 = \frac{1}{2}\begin{bmatrix} (f(\alpha_1) - y_1)^2 \\ (f(\alpha_2) - y_2)^2 \end{bmatrix}$$

$$\nabla_{\boldsymbol{W}}\boldsymbol{L} = \left(\nabla_{\hat{\boldsymbol{y}}}\boldsymbol{L} \odot f'(\boldsymbol{a})\right)\boldsymbol{x}^T = \left(\begin{bmatrix} \widehat{y_1} - y_1 \\ \widehat{y_2} - y_2 \end{bmatrix} \odot \begin{bmatrix} f'(a_1) \\ f'(a_2) \end{bmatrix}\right)[x_1 \quad x_2 \quad x_3] = \begin{bmatrix} \nabla_{w_{11}}L_1 & \nabla_{w_{21}}L_1 & \nabla_{w_{31}}L_1 \\ \nabla_{w_{12}}L_2 & \nabla_{w_{22}}L_2 & \nabla_{w_{32}}L_2 \end{bmatrix}$$

Derivation of general neural net – each layer + NN learning



- define:

  $\boldsymbol{h_0 = x}$

- compute forward propagation:

  for k = 1 to K

  $\boldsymbol{a_k = W_k\, h_{k-1}}$

  $\boldsymbol{h_k = f(a_k)}$

  note:

  $h_K = \hat{y}$

  $\nabla_{h_K} L = \nabla_{\hat{y}} L$

- compute gradient backpropagation:

  for k = K to 1

  $\nabla_{a_k} \boldsymbol{L} = \nabla_{h_k} \boldsymbol{L} \odot f'(\boldsymbol{a_k})$

  $\nabla_{h_{k-1}} \boldsymbol{L} = \boldsymbol{W_k^T}\, \nabla_{a_k} \boldsymbol{L}$

  $\nabla_{W_k} \boldsymbol{L} = \nabla_{a_k} \boldsymbol{L}\, \boldsymbol{h_{k-1}^T}$

- update weights (by vanilla SGD):

  $\boldsymbol{W^{(it+1)} = W^{(it)} - \mu \nabla_{W^{(it)}} L}$

  μ – learning rate

  it – iteration

Learning Algorithms

- Basics
  - principle of neural nets
  - single neuron
    - principle
    - activation functions
    - derivation
  - general neural net
    - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
  - hyperparameter optimization

Learning Algorithms – “Vanilla” GD

1. Gradient Descent (GD)
   - weight update after each epoch (one loss and gradient value for whole training dataset)
   - not used – slow, unfeasible for large training dataset, local extrema of the loss function are lost
   - for appropriate learning rate GD converges to the global minimum for convex loss functions, to the local minimum for non-convex loss functions
   - “online” update is no allowed

$$\boldsymbol{W}^{(epoch+1)} = \boldsymbol{W}^{(epoch)} - \mu \nabla_{\boldsymbol{W}^{(epoch)}} \boldsymbol{L}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{W}^{(epoch)})$$

2. Stochastic Gradient Descent (GD)
   - weight update after each iteration (one loss and gradient value for each sample of training dataset) – computational demanding, but faster than full GD
   - high variance of updates – fluctuation
   - enables jumping to better local extrema
   - overshooting the global extreme – can be solved by slow decreasing the learning rate
   - shuffle the training data randomly at the beginning of each epoch

$$\boldsymbol{W}^{(it+1)} = \boldsymbol{W}^{(it)} - \mu \nabla_{\boldsymbol{W}^{(it)}} \boldsymbol{L}(\boldsymbol{x}^{(it)}, \boldsymbol{y}^{(it)}, \boldsymbol{W}^{(it)})$$

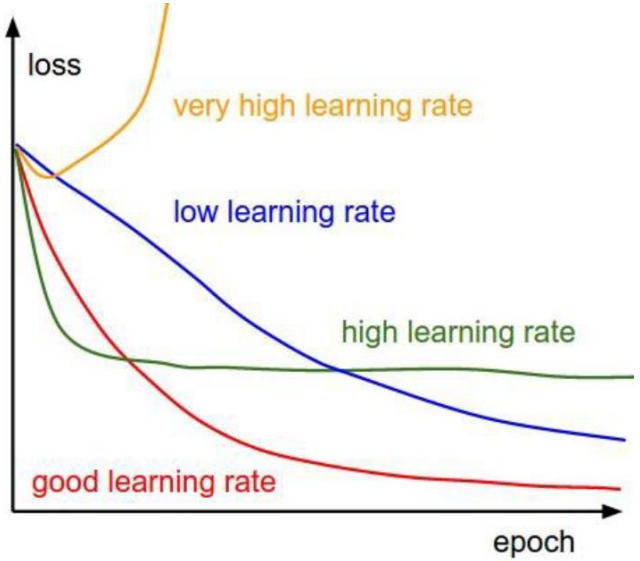
Learning Algorithms – “Vanilla” GD

3. Mini-batch Gradient Descent
   - “between” GD and SGD
   - weight update is done after few (n) iterations
   - reduces the variance of weight updates
   - enables efficient parallelization
   - n is in the range of tens to hundreds/thousands – depends on solved problem, size of training data, and available memory
   - commonly used algorithm

$$\boldsymbol{W}^{(it+1)} = \boldsymbol{W}^{(it)} - \mu \nabla_{\boldsymbol{W}^{(it)}} \boldsymbol{L}(\boldsymbol{x}^{(it:it+n)}, \boldsymbol{y}^{(it:it+n)}, \boldsymbol{W}^{(it)})$$

Learning Algorithms – “Vanilla” GD – problems

- chose a proper learning rate?
- learning rate schedules…
- same learning rate for all parameters?
- problem of saddle point plateau – same error in all directions -> zero gradient…
- problem with local extrema



Learning Algorithms – GD with learning rate modification

4. Adaptive Moment Estimation (Adam)
   - combines exponentially decaying average of past squared gradients $\boldsymbol{v}_{it}$ (such as Adadelta/RMSprop) and exponentially decaying average of past gradients $\boldsymbol{m}_{it}$ (such as momentum)
   - momentum -> behaves like a friction in Adam
   - nowadays, the most frequently used algorithm

$$\boldsymbol{m}^{(it+1)} = \beta_1 \boldsymbol{m}^{(it)} + (1-\beta_1)\nabla_{\boldsymbol{W}^{(it)}}\boldsymbol{L}$$
$$\boldsymbol{v}^{(it+1)} = \beta_2 \boldsymbol{v}^{(it)} + (1-\beta_2)\left(\nabla_{\boldsymbol{W}^{(it)}}\boldsymbol{L}\right)^2$$

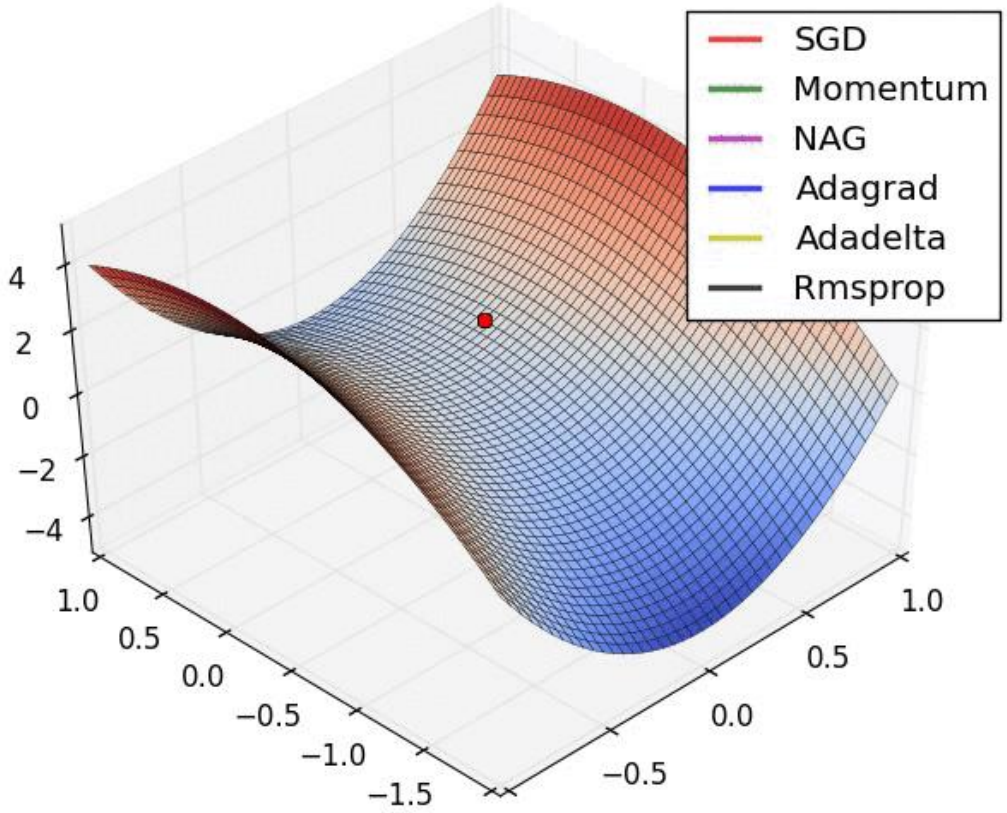
β_1 – decay rate of the first moment [0,1] – 0.9
β_2 – decay rate of the second moment [0,1] – 0.999
ϵ – smoothing term – arbitrary small – 10^{-8}

$$\boldsymbol{W}^{(it+1)} = \boldsymbol{W}^{(it)} - \frac{\mu}{\sqrt{\boldsymbol{v}^{(it)} + \epsilon}} \odot \boldsymbol{m}^{(it)}$$

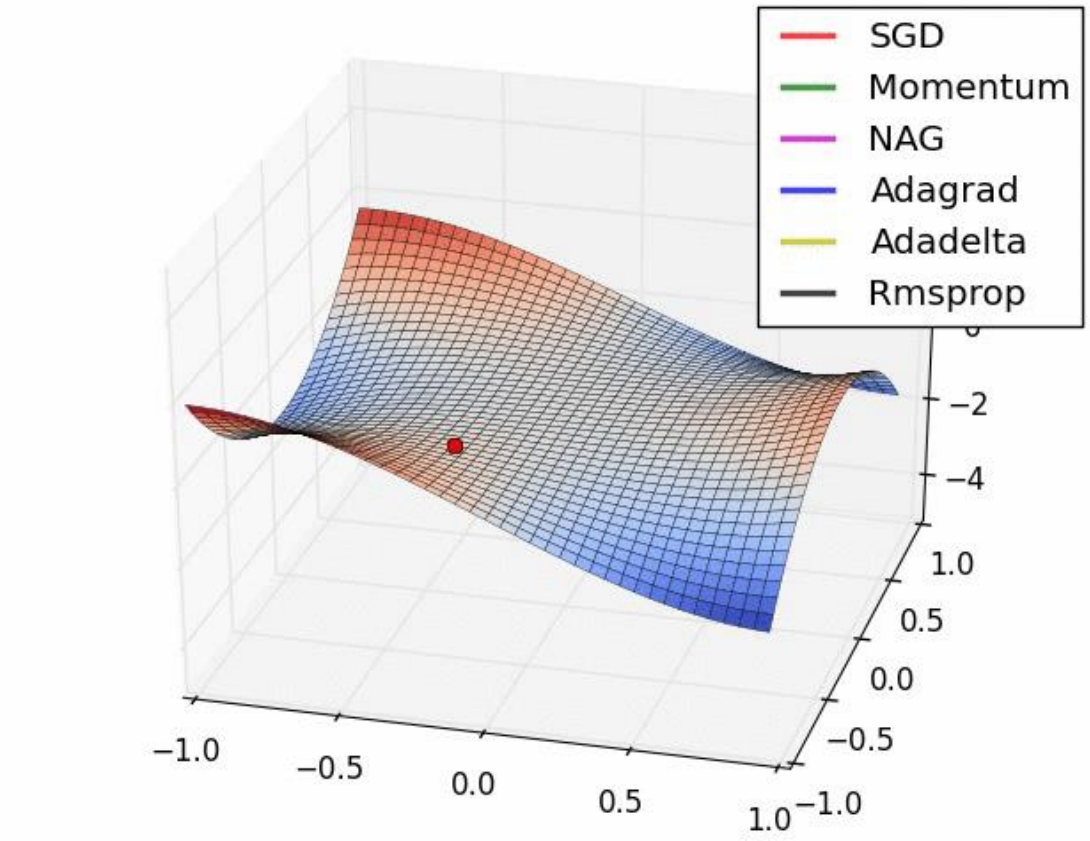
Learning Algorithms – GD with learning rate modification

- solves the problem with zero gradient
- solves the problem with local extrema
- solves the problem with initial learning rate
- solves the problem with learning rate for each parameter (weight)
- more additional hyperparameters (β_1, β_2)
- learning rate schedules…

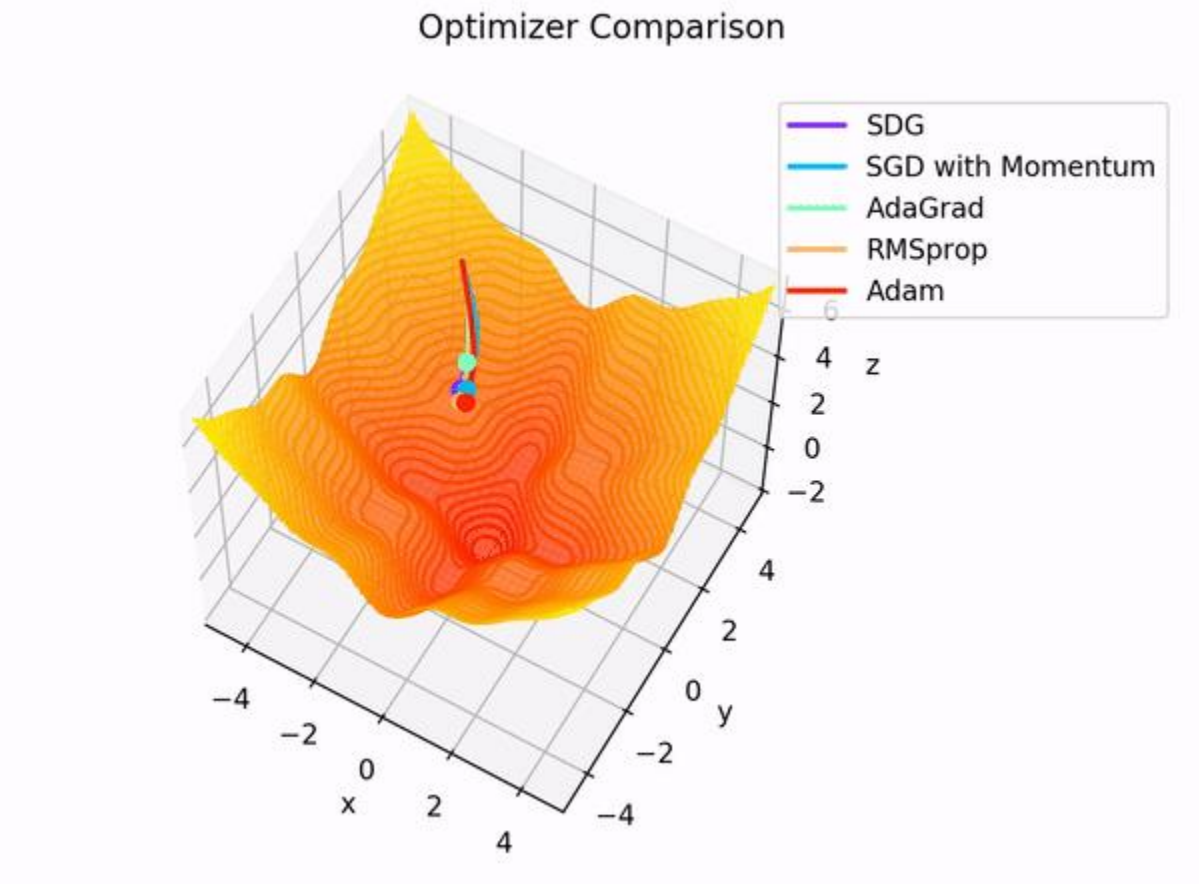
Learning Algorithms – examples



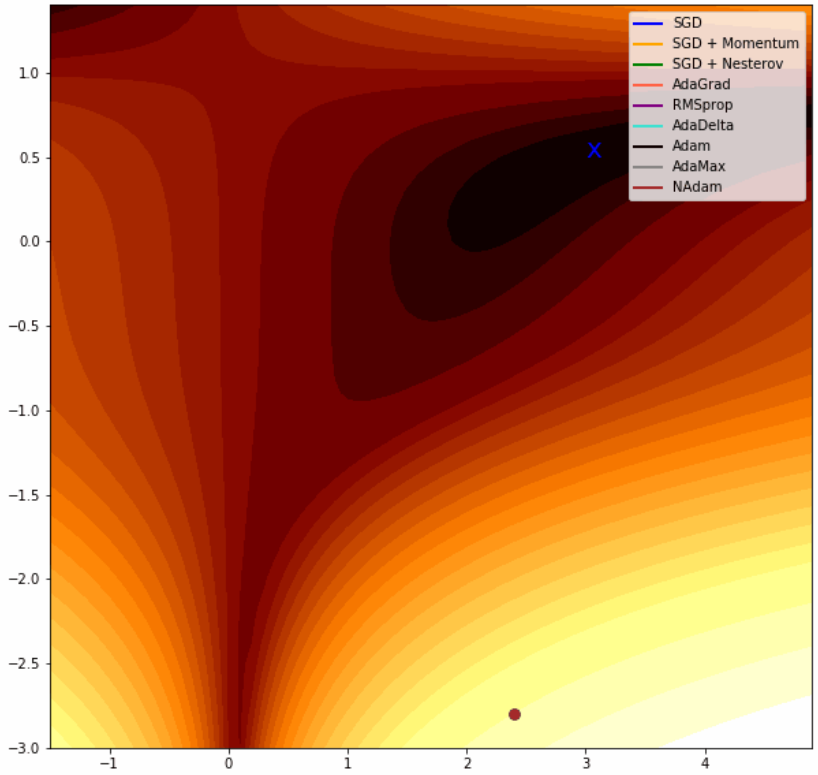
Learning Algorithms – examples



Learning Algorithms – examples



Learning Algorithms – examples



Learning Algorithms – Summary

- use mini-batch modifications of learning algorithms (if it is possible)
- use at least momentum or NAG – nowadays, Adam is standard
- use at least Adam if you don't want problems with the initial learning rate setting
- use learning rate schedules (even for β_1, β_2 in Adam)

Loss Functions

- Basics
  - principle of neural nets
  - single neuron
    - principle
    - activation functions
    - derivation
  - general neural net
    - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
  - hyperparameter optimization

Loss functions

Lp norm

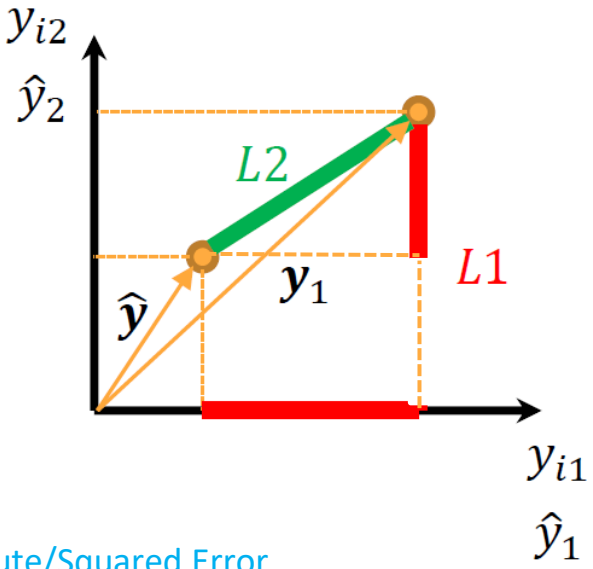
$$Lp = \|\boldsymbol{x}\|_p = \sqrt[p]{\sum_d |x_d|^p}$$

$$L1 = \|\hat{\boldsymbol{y}} - \boldsymbol{y}_1\|_1 = \sum_d |\hat{y}_d - y_{1d}| = |\hat{y}_1 - y_{11}| + |\hat{y}_2 - y_{12}|$$

$$L2 = \|\hat{\boldsymbol{y}} - \boldsymbol{y}_i\|_2^2 = (\hat{y}_1 - y_{11})^2 + (\hat{y}_2 - y_{12})^2$$

- Regression
  - L1 norm of the difference (Absolute Error)

$$L_i = \|\hat{y} - y_i\|_1 = |\hat{y} - y_i|$$



  - L2 squared norm of the difference (Squared Error)

$$L_i = \|\hat{y} - y_i\|_2^2 = (\hat{y} - y_i)^2$$

L1/L2 Loss

Mean Absolute/Squared Error

Total loss – for epoch/mini-batch

$$L = \sum_{i=1}^{N} L_i \quad \text{or} \quad L = \frac{1}{N}\sum_{i=1}^{N} L_i$$

Loss functions

Total loss – for epoch/mini-batch $L = \sum_{i=1}^{N} L_i$

- Classification
  - Multi-class Weighted Cross-entropy (WCE) Loss – for *one-hot encoded* classes – if sum of $\beta_c = 1$, balanced – classification problem

$$L_i = -\sum_{c=1}^{C} \beta_c y_{i,c} \log(\hat{y}_{i,c})$$

C – number of classes
c – class index
β_c – weight of class
$y_{i,c}$ – true class
$\hat{y}_{i,c}$ – predicted class (with Softmax activation function)

  - Binary class Tversky Loss – for segmentation problem

$$T = 1 - \frac{\sum_{i=1}^{N} y_{i,1}\hat{y}_{i,1}}{\sum_{i=1}^{N} y_{i,1}\hat{y}_{i,1} + \beta \sum_{i=1}^{N} y_{i,1}\hat{y}_{i,0} + \alpha \sum_{i=1}^{N} y_{i,0}\hat{y}_{i,1}}$$

TP TP FN FP

α, β – weights of trade-off between FP and FN
if $\boldsymbol{\alpha = \beta = 0.5}$ **– Dice coefficient (F1 score)**
if $\alpha = \beta = 1.0$ – Tanimoto coefficient (Jaccard)
if $\alpha + \beta = 1.0$ – higher β –> higher weight for FNs –> higher TPR

Regularization

- Basics
  - principle of neural nets
  - single neuron
    - principle
    - activation functions
    - derivation
  - general neural net
    - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
  - hyperparameter optimization

Regularization

- L1 and L2 regularization (Weight Decay) – see **Linear Regression topics**
  - regularization of weights of neural network during weights update
  - addition to loss function

$$\boldsymbol{W}^{(it+1)} = \boldsymbol{W}^{(it)} - \mu \nabla_{\boldsymbol{W}^{(it)}} (\boldsymbol{L} + \lambda \left\| \boldsymbol{W}^{(it)} \right\|_2^2)$$

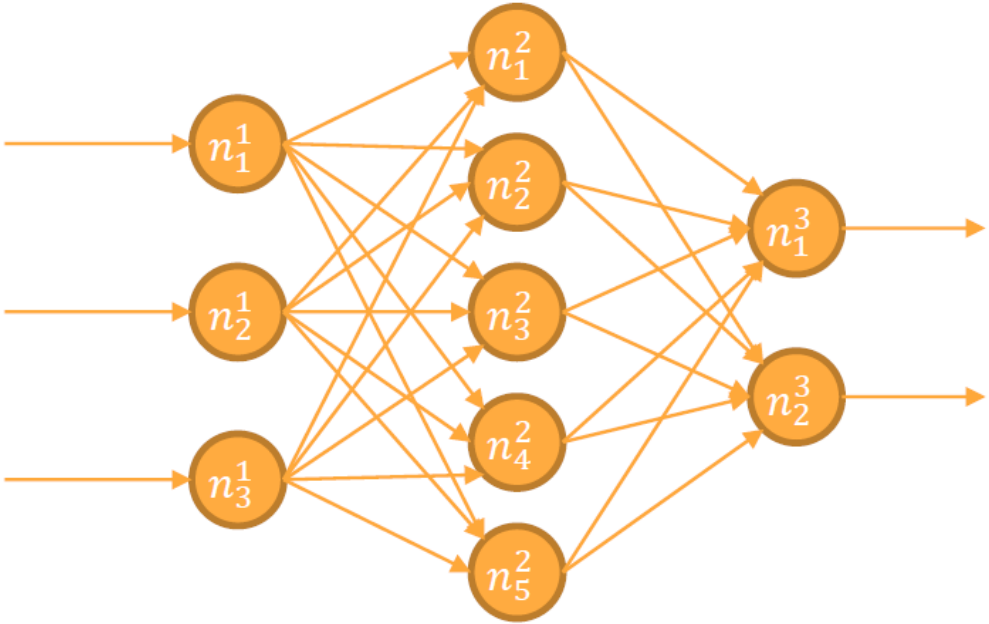
λ – regularization hyperparameter

Regularization – DropOut

- Motivation
  - We know, that combining of more different models improves the overall performance.
  - Also we know, that one complex model easily overfits and has high computational demands.
  - But, in the ANNs, we need to create many different architectures with unknown different hyperparameters, and ideally, we should train each model on different subset of training data. This is very complicated, time consuming and computationally demanding approach.

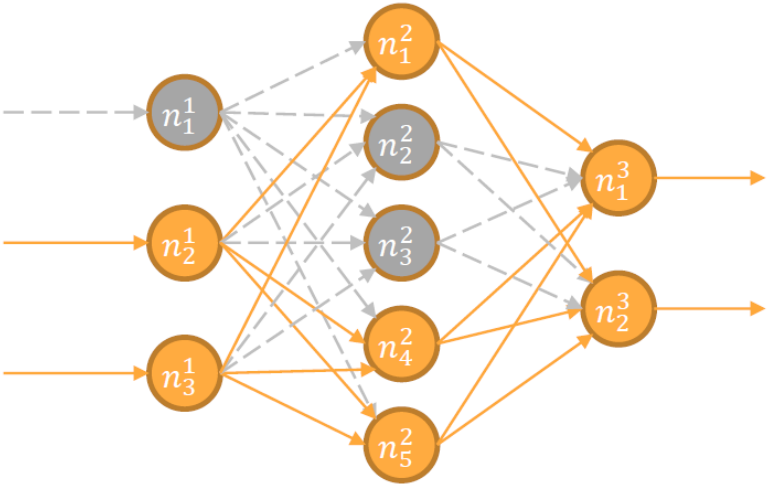
Regularization – DropOut

- What is it and how it works:
  - Full Neural Network



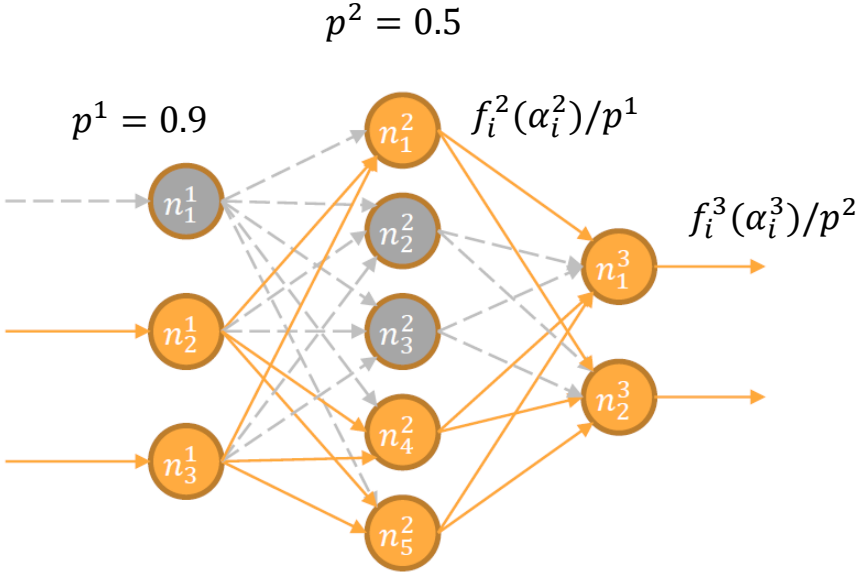
Regularization – DropOut

- What is it and how it works – Forward pass:
  - In each training epoch/mini-batch sample it randomly disables (“drops out”) some neurons in each layer in NN with all theirs input and output connections. DropOut state is given by Bernoulli random variable with given probability p for each neuron (1 = keep neuron, 0 = disable neuron, higher p means higher probability to be kept).
  - Combination in the last (output) layer – output neurons can’t be disabled – they define output class!



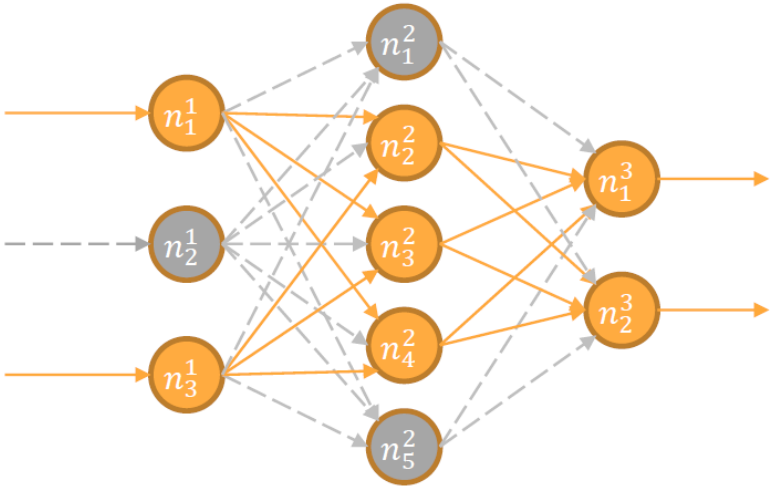
Regularization – DropOut

- What is it and how it works – Forward pass:
  - For this epoch/minibatch keep these neurons disabled
  - Output of each neuron in the next layer is divided by parameter p from previous layer – conservation of energy – especially important in deep architectures



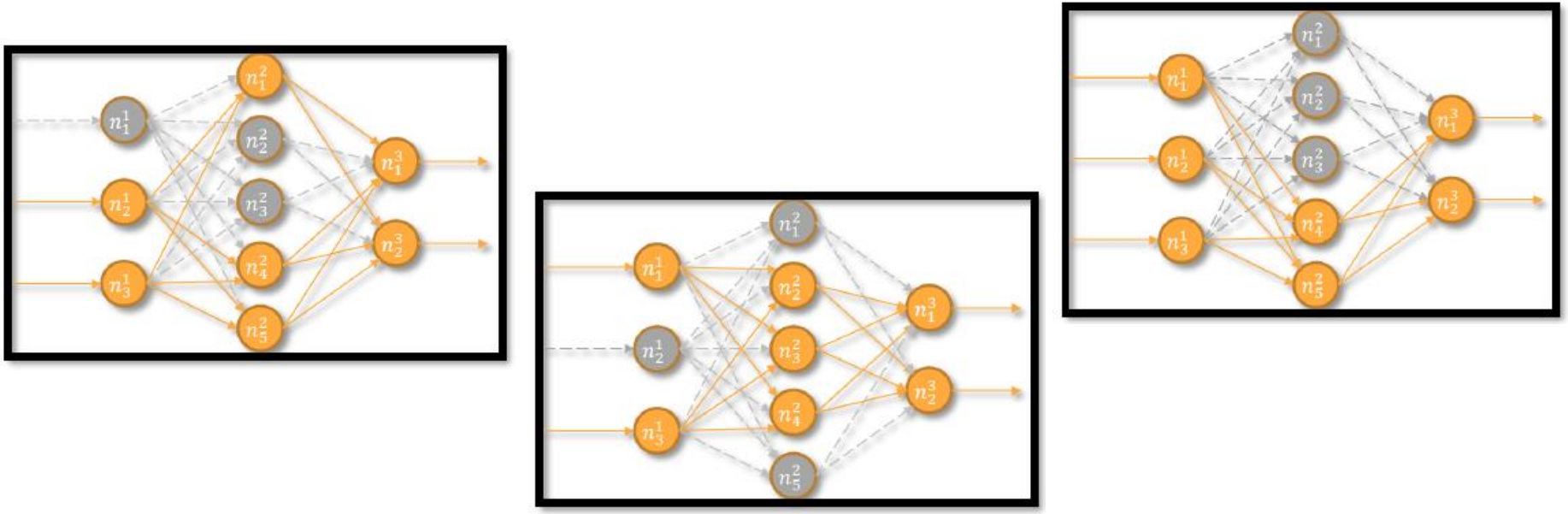
Regularization – DropOut

- What is it and how it works – Backward pass:
  - Compute loss and gradient only for neurons, that were not disabled
  - Update only weights of active neurons
  - Start new epoch/mini-batch: reactivate disabled neurons and continue training with a new realization of DropOut



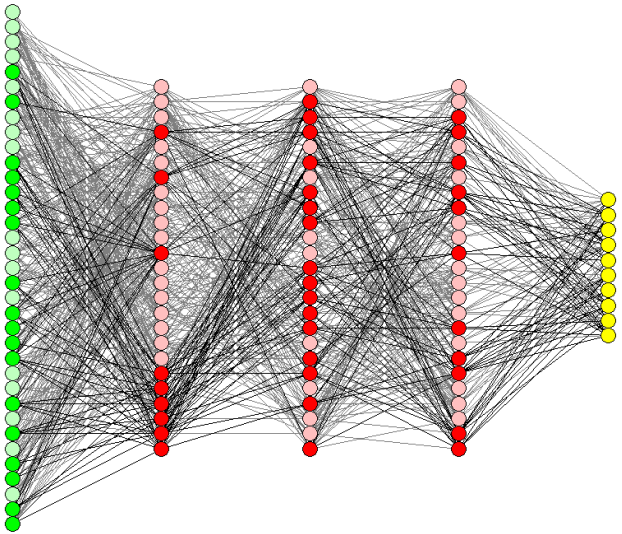
Regularization – DropOut

- What is it and how it works:
  - in each epoch/mini-batch a different small “thinned” sub-net is trained (regularization by reducing the complexity/size of model) – but they share their parameters (weights)



Regularization – DropOut

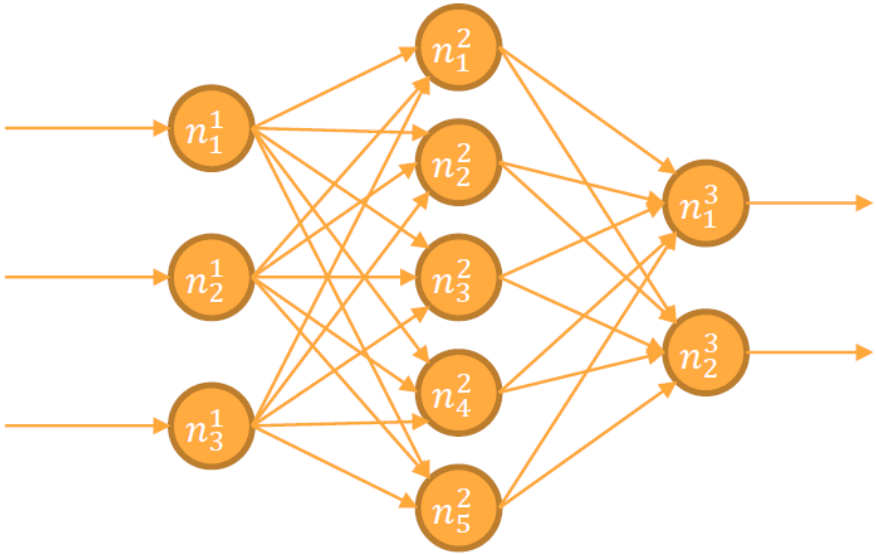
- What is it and how it works:
  - in each epoch/mini-batch a different small “thinned” sub-net is trained (regularization by reducing the complexity/size of model) – but they share their parameters (weights)



Gif by Michael Pearce

Regularization – DropOut

- What is it and how it works – Prediction/testing phase:
  - Combine all sub-nets together by activating all neurons!
  - Because the division of neural outputs by p was used during training, no other modification is needed – otherwise multiplication must be used in prediction phase!



Regularization – DropOut – Summary

- prevents overfitting
- approximation of “bagging” technique
- easy and simple
- fast to compute
- full trained NN approximately combines exponentially many different NN architectures – 2^N of possible sub-nets, where N is total number of neurons in the full NN
- thanks to weight sharing – still $O(n^2)$ parameters only
- can be (and should be) used together with L1/L2 regularization
- usual setting of p: for input layer near to 1.0, for hidden layers 0.5, for output layers 1.0!
- p can be also individually tuned
- could be ineffective if there are highly correlated features

Regularization – Batch Normalization

- normalization step that fixes means and variances of each layer's input
- changing normalization for each mini-batch and during training (because of changes of distribution of activations with weight changes and data changes)

$$D = \{(\boldsymbol{x}^{(1)}, y^{(1)}), \dots, (\boldsymbol{x}^{(N)}, y^{(N)})\}$$

1. compute mean and variance of mini-batch B for each feature/activation d separately – each mini-batch contains m samples of training data and store it for whole dataset

$$\mu_{B_d} = \frac{1}{m}\sum_{i-1}^{m} x_d^{(i)} \qquad \sigma_{B_d}^2 = \frac{1}{m}\sum_{i-1}^{m}\left(x_d^{(i)} - \mu_{B_d}\right)$$

$$\mu_{D_d} = \alpha\mu_{D_d} + (1-\alpha)\mu_{B_d}$$
$$\sigma_{D_d}^2 = \alpha\sigma_{D_d}^2 + (1-\alpha)\sigma_{B_d}^2$$

α – exponential decay

2. normalize mini-batch B according to these means and variances

$$\tilde{x}_d^{(i)} = \frac{x_d^{(i)} - \mu_{B_d}}{\sqrt{\sigma_{B_d}^2 + \epsilon}}$$

ϵ – arbitrary small constant

Regularization – Batch Normalization

3. transforms normalized data $\tilde{x}_d^{(i)}$ by learnable parameters scale γ_d and shift β_d: $\tilde{h}_d^{(i)} = \gamma_d \tilde{x}_d^{(i)} + \beta_d$
4. pass the batch normalized data $\tilde{h}_d$ to next network layers
5. by chain rule (BN is differentiable) train parameters scale γ_d and shift β_d
6. repeat 1. – 6. during training
7. AFTER TRAINING: calculate BN transformation parameters γ_d^{pred} and β_d^{pred} from point 3., that will be used in prediction phase as:

$$\tilde{h}_d^{pred} = \frac{\gamma_d}{\sqrt{\sigma_{D\,d}^2 + \epsilon}} x_d^{pred} + \left(\beta_d - \frac{\gamma_d \mu_{D\,d}}{\sqrt{\sigma_{D\,d}^2 + \epsilon}} \right)$$

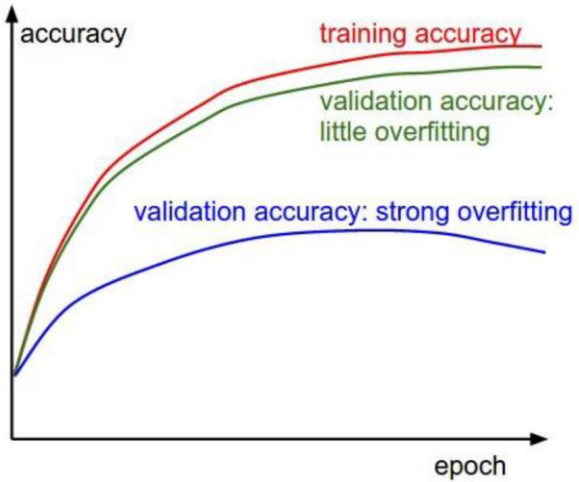
γ_d^{pred} β_d^{pred}

Regularization – Batch Normalization – Summary

- prevents overfitting
- nowadays is shown that BN performs better than DropOut (no need of DropOut and sometimes together with DropOut performs worse)
- single linear transform in prediction phase (no computational complexity)
- reduces dependency on initial LR and weights
- enables higher LRs
- enables lower L1/L2 regularization
- enables faster LR decay
- reduces the need of using pre-processing of the data
- speeds up training process
- needs appropriate size of the mini-batch – may be problem with a memory in huge models and/or size or dimensions of input data – not working with too small mini-batches
- dependent on mini-batch size – should be tuned

Regularization – another, but basic approaches

- limit the number of learning iterations/epochs
  - manually before training
  - manually during training
  - based on the number of validation checks
  - based on shape of learning curve (loss function changes during training)
  - comparing the shape of learning curves on training and validation dataset or the train/validation error gap (bias-variance trade-off)
- limit the size of mode
  - number if hidden layers
  - number of neurons
  - “bottlenecks”

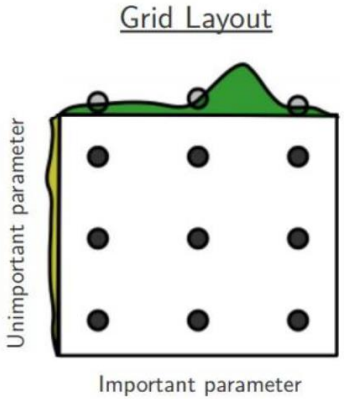


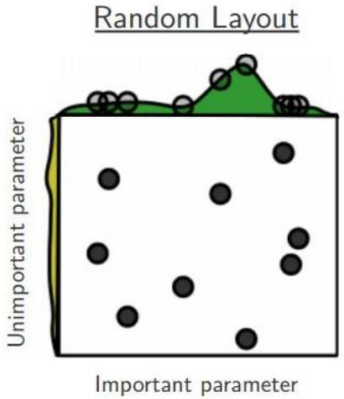
Tricks and Tips

- Basics
  - principle of neural nets
  - single neuron
    - principle
    - activation functions
    - derivation
  - general neural net
    - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
  - hyperparameter optimization

Tricks and Tips

- hyperparameter optimization
  - many hyperparameters that influence learning process and final validation performance ($\mu_0, \lambda, p, \alpha, k, minibatch\ size, \beta_1, \beta_2, etc.$)
  - each change of hyperparameter requires retraining of NN





👎 grid search – brute-force search may take several days/weeks/months

👉 random search – better than grid search
- can be improved by coarse to fine search
- be careful on borders
- may performs better than other if search intervals are chosen appropriately

👍 evolution algorithms – see MPC-EAL

👍 Bayesian optimization – Hyperopt, Spearmint, SMAC